Smart Parking Slot Detection

Authors: S.S.Hrithi Shree

N.Harshini

Date: 30-08-2025

College: Saranathan college of Engineering, Trichy

Department: Electronics and Communication Engineering

Abstract

- Efficient parking management is a challenge in urban and institutional environments. This project presents a **real-time smart parking system** that detects and classifies parking slots as **occupied, empty, or in transition**, using the **YOLOv8n (Nano) object detection model**. The system analyzes video footage frame-by-frame, generates structured **JSON slot data**, and visualizes it in a **React-based interactive dashboard**. The dashboard includes live video, **2D slot grid, pie charts, summary table, and map view**, providing actionable insights for parking management. YOLOv8n ensures **fast inference and lightweight deployment**, suitable for real-time applications even on low-resource systems.This work is motivated by increasing traffic congestion and inefficient parking management in smart cities making it suitable for real-time applications even on low-resource systems, with scalability for hackathon-ready and smart city deployments.

1. Introduction

Manual parking monitoring is inefficient and error-prone. Automating the detection and monitoring of parking slots can save time and improve space utilization. The system presented here combines **YOLOv8n object detection**, **frame-wise JSON mapping**, and **React visualization** to provide a **real-time,**

interactive dashboard. YOLOv8n was chosen because of its lightweight design, faster inference speed, and suitability for rapid hackathon deployment compared to heavier models

Objectives:

- Detect vehicles in real-time using YOLOv8n.
- Classify slots as **Occupied**, **Empty**, or **Transition**.
- Provide multiple views for monitoring: 2D grid, pie chart, summary table, and map.
- Ensure lightweight, fast, and scalable deployment.

2. System Overview

The system consists of four main stages:

1. **Input Video Acquisition** – Capturing the parking area with a mobile camera.
2. **YOLOv8 Object Detection** – Detecting vehicles frame-by-frame.
3. **JSON Slot Mapping** – Converting YOLO outputs into structured slot-wise data.
4. **React Dashboard Visualization** – Displaying live slot occupancy via video, 2D grids, charts, summary tables, and maps.

3. Data Acquisition

Feature	Description
Location	Saranathan College of engineering Parking Area
Camera	Vivo Y16 Mobile Phone
Time	Evening, outdoor
Scenario	Vehicles arriving/leaving; occlusion by trees
Duration	1 minute 24 seconds (1532 frames)
Format	MP4

4. YOLOv8n Object Detection

4.1 Model Overview

- YOLOv8n (Nano) is a **lightweight version of YOLOv8** optimized for speed and low memory usage.
- Suitable for real-time applications on laptops or embedded devices.
- Architecture: CSP backbone, PAN neck, YOLO detection head.

4.2 Dataset Preparation

- Custom parking dataset: images annotated with **vehicle bounding boxes** and corresponding **parking slots**.
- Augmentation techniques:
    - Brightness/contrast adjustment
    - Occlusion simulation
    - Rotation and scaling
- The dataset consisted of ~4000 annotated images (5 subsets × ~800 images each).Dataset split: 80% training, 10% validation, 10% testing.

4.3 Training

- Model: yolov8n.pt pre-trained weights (COCO dataset)
- Training parameters:
    - Epochs: 100
    - Batch size: 16
    - Learning rate: 0.001
    - Input size: 640×640
- Optimizer: **Adam**
- Loss functions: Objectness loss, classification loss, bounding box loss
- Hardware: NVIDIA GPU (if available) for faster training
- Final model achieved mAP50 ≈ 0.89 and mAP50-95 ≈ 0.72, demonstrating strong detection accuracy.

4.4 Inference

- Each video frame is passed to YOLOv8n.
- Outputs bounding boxes, class label, and confidence score for detected vehicles.
- Frame-wise output example (JSON):

[{"frame_idx": 120, "slot_id": "slot1", "status": "Occupied", "confidence": 0.92}, {"frame_idx": 120, "slot_id": "slot2", "status": "Empty", "confidence": 0.95}]

5. Slot-wise Dataset Creation

Convert YOLO outputs to **slot-level occupancy**:

  - slot_status.json → tracks Occupied/Empty/Transition per frame
  - slot_framesX.json → stores 2D coordinates of each slot

- Apply **majority voting across frames** to reduce flickering and false positives.
- This structured data feeds the React dashboard for visualization.

6. Dashboard Architecture

Component	Functionality
App.jsx	Loads video and JSON; manages current frame; synchronizes all components
VideoPlayer.jsx	Renders video and triggers frame updates
SlotGrid2D.jsx	Displays 2D parking grid; color-coded by slot status
MapView.jsx	Leaflet map showing slot markers with occupancy status
Charts.jsx	Pie chart and radial chart displaying slot occupancy distribution

Dashboard Features:

- Real-time synchronization of video, 2D grid, charts, and summary table
- Interactive map view for visualizing slot locations
- Responsive layout with pastel color scheme for clarity
- Supports multi-frame updates to reflect live changes

7. Implementation Details

7.1 Preprocessing

- Video frames extracted at 30 fps
- Resized and normalized to 640×640 for YOLOv8n input

7.2 Postprocessing

- Map vehicle detections to parking slots using bounding box overlap
- Slot occupancy determined as **Occupied, Empty, or Transition**
- JSON files generated for dashboard consumption

7.3 Dashboard Implementation

- Tech Stack: **ReactJS, Recharts, React-Leaflet**
- State management ensures synchronization across all components
- Dynamic charts and 2D grid update per frame

8. Challenges and Solutions

Challenge	Solution
JSON naming errors	Standardized JSON file names and structure
2D slot grid rendering issues	Corrected slot coordinates and state management
Pie chart synchronization	Updated currentSlots on each video frame

Map zoom/alignment issues	Centered map with appropriate zoom
Video alignment and size	Adjusted flexbox layout for proper alignment

9. Model Performance

- **Accuracy:** YOLOv8n achieves high precision for occupied vs. empty slot detection
- **Speed:** Lightweight, supports near real-time processing (~25–30 fps on GPU, 10–15 fps on CPU)
- **Robustness:** Handles occlusion, varying lighting, and dynamic vehicle movements
- **Output:** Annotated video + CSV/Excel logs for all slots

10. Results

- Fully functional **Smart Parking Dashboard**
- Live **video with dynamic slot updates**
- Dynamic **summary table video** and **2D grid + pie chart video**
- Map view visualizing slot positions and occupancy
- Real-time charts and multiple views for easy monitoring

METRIC	Value
Accuracy	95%
Precision	97%
Recall	92%
FPS(GPU)	25 - 30
FPS(CPU)	10 - 15

11. Conclusion

- The project demonstrates a **robust, lightweight, and scalable smart parking system** using **YOLOv8n**. Integration with a React dashboard enables **real-time slot monitoring**, dynamic visualization, and actionable insights for parking management. YOLOv8n ensures **fast inference** while maintaining high accuracy, making it suitable for practical deployment. Future enhancements include integration with IoT sensors for hybrid detection, deployment on Raspberry Pi / Jetson Nano for edge computing, and cloud dashboards for city-wide smart parking management.A heatmap analysis of frequently occupied slots and duration tracking per vehicle can further improve usability.